

DTE Platform — Engineering Standards & Development Constitution

As the Principal .NET Architect and Engineering Excellence Lead, I present the definitive Engineering Constitution for the V1 DTE Platform. This document is the unyielding law governing all code contributions, pull requests, and architectural decisions. It is designed to protect the platform's multi-year lifespan, regulatory compliance, and audit defensibility. Do not attempt to bypass these rules. They exist to protect the legal entity operating this software.

1. Engineering Philosophy

The V1 DTE Platform is governed by absolute principles prioritizing regulatory truth over developer convenience.

- **Compliance-Driven Engineering:** The software acts on behalf of the legal entity. It enforces rules defined by the Ministry of Finance (MH); it does not invent them.
- **The Snapshot Law:** A validated fiscal document is a photograph of facts in time. Master data must be copied entirely by value into immutable structures. Retroactive corruption via live data updates must be physically impossible.
- **Evidence Preservation over Storage Optimization:** History is immutable. Evidence artifacts and audit entries are append-only. We never overwrite, normalize, or delete regulatory artifacts to save disk space.
- **Domain Integrity over Convenience:** Domain invariants are absolute. If enforcing an invariant requires more code, write the code. We do not compromise the state machine to save keystrokes.
- **Explicitness over Magic:** Avoid reflection, dynamic typing, and hidden event cascades. The execution path of a fiscal document must be easily traceable by an auditor.

2. Solution Structure Rules

The solution strictly enforces Clean Architecture using compiler-level project references (.csproj).

- **Project Boundaries:** The solution consists of exactly four core assemblies: DTE.Domain, DTE.Application, DTE.Infrastructure, and DTE.Api.
- **Layer Responsibilities:** Domain owns invariants. Application owns orchestration and CQRS handlers. Infrastructure owns EF Core, PostgreSQL, Hangfire, and MH integration. Api owns HTTP routing.
- **Dependency Direction:** Dependencies flow strictly inward. Api depends on Application. Infrastructure depends on Application. Application depends on Domain. Domain depends on nothing.
- **Clean Architecture Enforcement:** NetArchTest unit tests run in the CI pipeline. Any commit violating dependency directions will fail the build immediately.
- **CQRS Boundaries:** State-mutating Commands must be physically isolated from data-retrieval Queries. They do not share DTOs or MediatR handlers.

3. Domain Layer Standards

The DTE.Domain assembly is the heart of the regulatory engine.

- **Aggregate Design Rules:** Aggregate boundaries define transactional consistency. Everything inside a root is changed together. Cross-aggregate references are strictly by identity (UUID) only.
- **Entity Rules:** Entities have an identity and lifecycle but live entirely inside an aggregate root. They cannot be queried or persisted independently of their root.
- **Value Object Rules:** Value objects are immutable, identity-less, and compared by value. They cannot be edited in place.
- **Domain Event Rules:** Domain Events record facts in the past tense (e.g., DocumentValidated). They must not orchestrate external API calls directly.
- **Invariant Enforcement:** Invariants represent the constitution of the domain. Document Totals must perfectly equal the sum of Document Lines plus the Tax Summary before a document can leave the Draft state. Treat any violation as a domain-level exception.
- **State Machine Rules:** Transitions must be exhaustive and strictly enforce the Draft -> Validated -> Signed -> Submitted -> Accepted/Rejected flow. Skips and undefined reversals are strictly forbidden.

4. Application Layer Standards

The DTE.Application assembly orchestrates the domain using MediatR.

- **MediatR Usage:** Every Use Case is implemented as a single MediatR IRequestHandler.
- **Command Handler Standards:** Commands mutate state. They must return minimal acknowledgment or the resulting Aggregate ID. They must delegate invariant checking to the Domain.
- **Query Handler Standards:** Queries bypass domain logic entirely to provide highly performant, read-only data access directly from PostgreSQL JSONB snapshots.
- **Validation Rules:** Structural validation (e.g., missing fields) is handled via FluentValidation pipeline behaviors before the command reaches the handler. Domain validation (e.g., totals mismatch) is handled inside the Domain.
- **Pipeline Behaviors:** Cross-cutting concerns (Logging, Validation, Audit Generation) must be implemented as MediatR IPipelineBehavior classes to keep handlers clean.
- **Transaction Boundaries:** Transactions are strictly bound to the Aggregate Root. Cross-aggregate consistency is achieved through asynchronous side-effects, never through distributed or sprawling database transactions.

5. Infrastructure Standards

The DTE.Infrastructure assembly manages PostgreSQL and Hangfire.

- **EF Core Conventions:** Configuration must be managed via IEntityTypeConfiguration classes, keeping the DbContext clean.
- **PostgreSQL Conventions:** UUIDs are the standard primary key. NUMERIC(18,4) must be used for all financial precision.
- **JSONB Mapping Conventions:** IssuerSnapshot, ReceptorSnapshot, and ItemSnapshot must be mapped to PostgreSQL JSONB columns natively using EF Core.

- **Migration Strategy:** Migrations must be script-based, generated via EF Core, and strictly reviewed for destructive operations. Dropping columns requires explicit Architectural Board approval.
- **Repository Implementation Rules:** Repositories must load and save entire Aggregate Roots. "Partial updates" or "generic repositories" (Repository<T>) are strictly forbidden.

6. API Standards

The DTE.Api assembly is a thin HTTP wrapper.

- **Endpoint Naming:** Use RESTful noun-based routing (e.g., POST /api/v1/documents, PUT /api/v1/documents/{id}/validate).
- **Versioning Strategy:** URI versioning (/v1/) is mandatory from day one.
- **Error Response Format:** Standardized RFC 7807 Problem Details for HTTP APIs.
- **Validation Response Format:** HTTP 400 Bad Request containing a flat array of explicit property-level error messages.
- **Idempotency Requirements:** All Commands resulting in external side effects (e.g., MH transmission) must require an Idempotency-Key header. Repeating a command with the same key must yield the same resulting state without duplicating external effects.

7. Security Standards

Customer sovereignty and least privilege dictate the security model.

- **Authentication Standards:** Stateless JWT issued upon login, delivered strictly via HTTPOnly, SameSite=Strict cookies to prevent XSS.
- **Authorization Standards:** Strict Role-Based Access Control (RBAC). Endpoints must demand exactly Owner or Biller via declarative attributes.
- **Secrets Management:** MH API credentials and connection strings must be injected via environment variables or secure vaults, never committed to source control.
- **Certificate Handling:** The application NEVER loads the .pfx or private key directly into application memory. It utilizes a certificate_reference to request signatures from the secure local vault.
- **Audit Requirements:** The actor's UUID and role must be securely extracted from the validated JWT for audit attribution.

8. Audit & Evidence Standards

Defensibility is non-negotiable.

- **Evidence Immutability Rules:** Evidence artifacts (Payload, Stamp, PDF) are inserted into BYTEA columns. Database roles must physically lack DELETE and UPDATE privileges on these tables.
- **Append-Only Requirements:** Transmission records and invalidation events are append-only.
- **Audit Generation Requirements:** Every state-mutating command must natively produce a tamper-evident audit entry. If the audit entry fails to persist, the Command must fail.
- **Hash-Chain Requirements:** The CurrentHash of the audit entry must mathematically embed the PreviousHash, ActorIdentity, Action, and Timestamp. Breaking the chain must be detectable.

9. Testing Constitution

Tests must prove regulatory correctness and architectural integrity.

- **Unit Testing Standards:** xUnit and FluentAssertions. Focus on Value Objects, Totals reconciliation, and pure Domain logic.
- **Integration Testing Standards:** Must use Testcontainers (Docker-in-Docker) to spin up a real PostgreSQL database.
- **Architectural Testing Standards:** NetArchTest rules must run on every build to prevent forbidden dependencies.
- **Testcontainers Requirements:** Do not use Entity Framework In-Memory providers. They do not enforce JSONB constraints, Triggers, or accurate transactional locking.
- **Forbidden Mocking:** You cannot mock the Database during integration testing. You cannot mock the Domain during Application testing. Mock only external APIs (MH) and Vaults.

10. Logging & Observability Standards

Logging must provide operational insight without leaking PII or regulatory secrets.

- **Structured Logging:** Serilog must be used to output JSON-formatted logs to the filesystem and stdout.
- **Correlation IDs:** Every incoming HTTP request must generate a Correlation ID, passed into MediatR, and appended to all Serilog events and Hangfire jobs.
- **Audit Correlation:** Operational logs must include the document_id and generation_code where applicable, without logging the actual JSON payload (which belongs in the secured Evidence table).
- **Background Job Tracing:** Hangfire job execution logs must tie back to the original HTTP request Correlation ID that enqueued them.

11. Error Handling Standards

Errors must be cleanly categorized and translated at the API boundary.

- **Domain Exceptions:** Thrown by the Domain for invariant violations (e.g., InvalidDocumentStateException). Mapped by the API to 422 Unprocessable Entity.
- **Application Exceptions:** Thrown by MediatR pipelines for missing data or structural issues. Mapped to 400 Bad Request or 404 Not Found.
- **Infrastructure Exceptions:** Thrown by DB or network failures. Mapped to 500 Internal Server Error (or 502 Bad Gateway for MH API timeouts).
- **Regulatory Validation Failures:** Explicit MH catalog failures (e.g., unsupported DTE type) map to 409 Conflict.

12. Naming Conventions

The Ubiquitous Language is absolute.

- **Projects:** DTE.Domain, DTE.Application, DTE.Infrastructure, DTE.Api.
- **Namespaces:** Grouped by feature slice, not technical concern (e.g., DTE.Domain.Issuance, DTE.Application.Contingency).
- **Classes & Aggregates:** Must perfectly match the canonical domain spec

(FiscalDocument, ControlNumberSeries, TransmissionRecord).

- **Commands & Queries:** Must express intent (ValidateDocumentCommand, GetEvidenceBundleQuery).
- **DTOs:** Postfixed with Request or Response (e.g., DocumentValidationRequest).
- **Database Objects:** Lower snake_case for PostgreSQL (e.g., fiscal_document, evidence_bundle).

13. Code Review Constitution

A Pull Request must be instantly rejected if it fails any of the following checks.

Review Checklist Requirement	Enforcement Criteria
Architectural Boundaries	NetArchTest passes; no cross-layer bleed.
Ubiquitous Language	No synonyms used in code, variables, or tests.
Domain Isolation	Domain models have zero EF Core, JSON, or framework attributes.
Audit Coverage	State mutation commands correctly trigger AuditEntry generation.
Database Immutability	Modifying Evidence or Audit triggers DB-level rejection.
Snapshot Law Verification	DTOs map to JSONB; live master data is not referenced via FKs.
Idempotency	Retry mechanisms on MH integrations do not duplicate side-effects.

14. AI Development Rules

AI tools (Gemini, GPT, Claude) are treated as junior developers lacking architectural context.

- **What AI May Generate:** Boilerplate code, Unit Test scaffolds, Regex, JSON schema validation logic, and repetitive EF Core mappings.
- **What AI May Not Redesign:** AI is strictly forbidden from altering the Domain Model, API Contract, or Database Schema.
- **Required Human Verification:** A human engineer is legally responsible for verifying that AI-generated Domain logic perfectly enforces the invariants defined in the Canonical

Domain Specification.

15. Technical Debt Policy

We distinguish between strategic deferral and architectural rot.

- **Acceptable Debt:** Hardcoding a simple UI layout temporarily; deferring performance tuning on a low-traffic query; delaying the automated test coverage of an edge-case UI component.
- **Unacceptable Debt:** Bypassing an audit log because "it's hard to wire up"; putting business logic in a Controller to save time; mocking the database because Testcontainers are "slow".
- **Refactoring Expectations:** Refactoring must happen within the confines of the Domain Specification. Refactoring that alters the API Contract or Database Schema requires explicit Architecture Board approval.

16. Forbidden Practices

The following patterns represent immediate grounds for Pull Request rejection and architectural remediation.

- **Anemic Domain Models:** Domain classes that contain only properties (`{ get; set; }`) with all logic stripped out and placed in Application Services.
- **Business Logic in Controllers:** Controllers doing anything more than parsing HTTP requests, calling MediatR, and returning an HTTP status code.
- **Direct DbContext Usage Outside Infrastructure:** Exposing `DbSet<T>` or EF Core anywhere in the Application or Domain layers.
- **Mutable Fiscal Evidence:** Any code path that updates or deletes a row in `evidence_bundle` or `evidence_artifact`.
- **Live Joins into Signed Documents:** Using PostgreSQL foreign keys to link a `FiscalDocument` to a live `Counterparty` table.
- **Bypassing Audit Generation:** Writing direct SQL or scripts to modify state without traversing the MediatR command pipeline that guarantees hash-chain generation.
- **Shared Transactions Across Bounded Contexts:** Opening a transaction that updates `FiscalDocument` and `Item` master data simultaneously. Aggregates are consistent strictly within their own boundaries.